Adobe University Hackathon 2026

Project: Chameleon Challenge

Adobe

CHAMELEON

Any Design. Every Canvas.

The Agentic Brand-Studio Challenge

Build the engine that resizes any design to any canvas — and keeps it looking designed.

Adobe University Hackathon: North America 2026

---

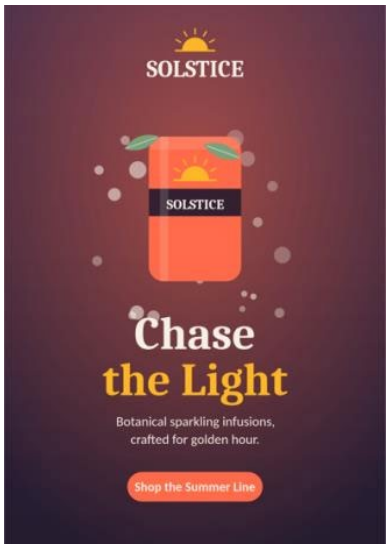
1. Introduction

Every brand lives on many screens at once. A single idea – a product launch, a sale, an event – has to appear as a square social post, a tall phone story, a wide website banner, a printed poster, and a dozen shapes in between. Today a designer rebuilds that idea by hand for each shape, one canvas at a time. It is slow, repetitive work, and it is the same work over and over.

A problem relatable to all of us: Suppose we have created an amazing brand asset for Adobe University Hackathon and our branding team has created an asset in portrait shape. We are happy with the design, but now we want it in landscape.

Before going further, it's worth grounding what we mean by a design. A poster – or any design – is made of a handful of ingredient parts: text, images, and other design elements, arranged with intent to convey a particular message. Those elements are placed on an otherwise empty canvas, and that arrangement is what creates an appealing, on-brand message.

Easy job to do – Right? Wrong, let me show you what happened when I tried doing the same on Express.



Original poster — Solstice "Chase the Light," 1080×1350 (portrait).

Original poster — element breakdown

Label	Element	Content

BG Image	Background artwork	Dark gradient with a decorative dot pattern
Image	Product art	Solstice bottle illustration with sun icon and logo band
Text	Headline	"Chase the Light"
Text	Subheading	"Botanical sparkling infusions, crafted for golden hour."
Text	CTA button	"Shop the Summer Line"



Express workflow — resizing to Instagram square via Express's Resize tool.

What happens at 1080 × 1080



Note: See the important brand elements that were lost from the top and bottom – no sane marketing team would use this feature.

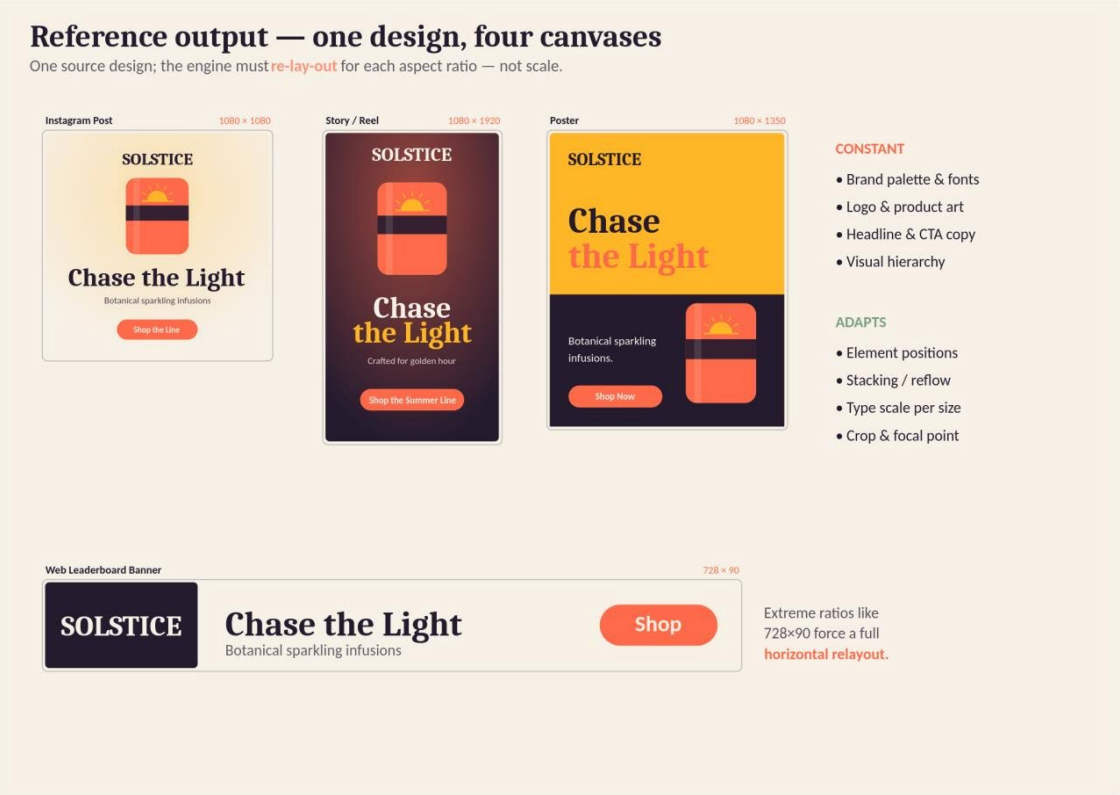
Opening Novel Ideas to a Problem We Tackle Daily

Project CHAMELEON asks student teams to build the engine that does this automatically. Give it a finished design and a new canvas size, and it should produce a new version of that design that fits the new shape and still looks like a designer made it – nothing overlapping, text readable, the brand intact, everything arranged with intent.

The name is the brief. A chameleon changes its appearance to suit any environment while remaining, unmistakably, the same animal. The engine must reshape a design to suit any canvas while keeping it, unmistakably, the same design.

Why this is exciting for Adobe. This is a real, still-unsolved product problem at the heart of Adobe Express: intelligent, automatic resizing of real designs.

A strong entry is a working glimpse of an agentic design assistant. Anyone who has ever resized a design by hand understands the value in a sentence.



One idea across several canvases. What stays the same (brand, content, hierarchy) and what must adapt (positions, stacking, type size, cropping) when the shape changes.

2. The challenge

Build a system that takes an existing design and a target canvas size, and automatically produces a new version of that design laid out for the new size – with no human touching it. The system must rearrange the design, not simply shrink or stretch it.

That distinction is the whole challenge. Imagine moving furniture into a room of a different shape. You don't photograph the old room and stretch the picture to fit – the sofa would look absurd. You keep the same furniture and place it again: the sofa moves to the long wall, the lamp to the corner, the rug is turned. A naive program stretches the picture; a good one moves the furniture. We are asking teams to move the furniture.

2.1 What teams build

The engine — the part we score. A standalone service that receives a design and a target size and returns instructions for the new layout. It does the hard thinking. It has no screen and no buttons; it simply takes input and returns output, so it can be tested automatically at scale.

2.2 What goes in

The engine receives the design in any particular size. The design arrives as its list of elements – each a piece of text, an image, or a shape – with properties such as position, size, colour, font, stacking order, and grouping.

2.3 What comes out

The engine returns the design in the same structure it received, but with updated properties – new positions, sizes, and so on – for the target canvas. In other words, it hands back a resized description of the same design. Our renderer then applies those properties to the original elements to produce the final result.

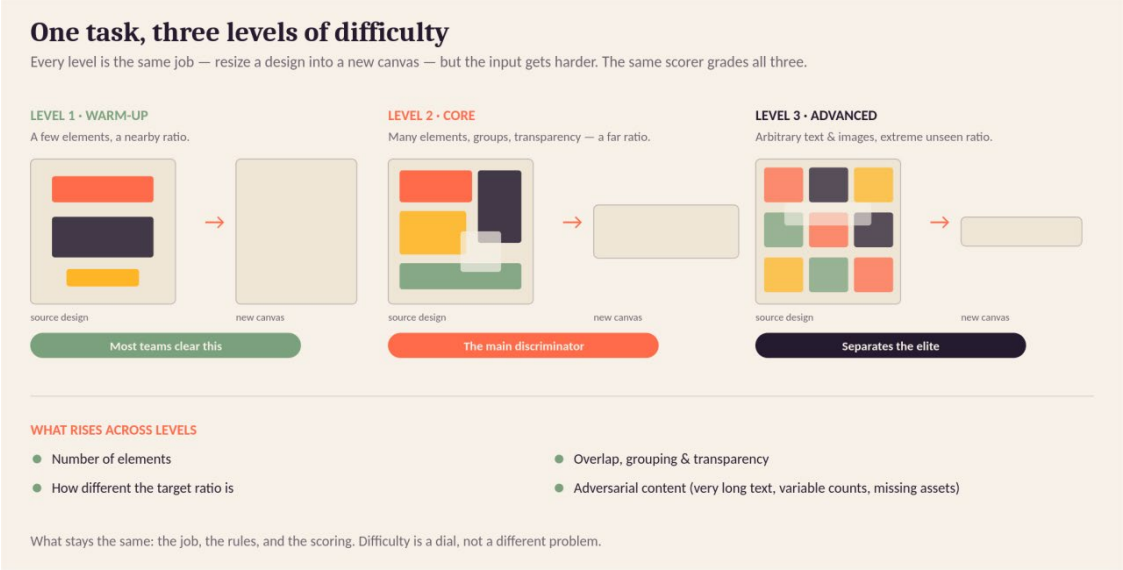
A short, illustrative example:

```
target canvas: 1080 x 1920 (tall)
returned design (same elements, new properties):
  logo -> x:380 y:120 w:320 h:120
  product -> x:324 y:300 w:432 h:720 (kept in proportion)
  headline -> x:90 y:1120 w:900 size:150 wraps to 2 lines
  cta -> x:240 y:1560 w:600 h:120
```

A response schema will specify exactly what the engine must return; it is being defined alongside the input schema.

2.4 One task, three levels of difficulty

The job is always the same – resize a design into a new canvas. What changes is how hard the input is. The three levels are graded the same way, so difficulty is a dial we turn, not a different problem.



- **Level 1 —> Warm-up.** A simple design with a few elements, resized to a nearby shape.
- **Level 2 —> Core.** A complex design – many elements, grouped pieces, overlapping and semi-transparent shapes – resized to a very different shape, such as a square turned into a wide banner.
- **Level 3 —> Advanced.** Designs with an arbitrary number of text boxes and images, deliberately awkward content, and an extreme target shape no team has seen before.

This is a hard problem — and that is the point. Automatic resizing of complex designs is not fully solved even inside Adobe; the middle level already goes beyond what today's tools do reliably. We are not looking for perfection – we are looking for how far a team can climb. A thoughtful, partial solution is genuinely valuable, and teams would be encouraged to submit even if their approach feels simple. No one should self-select out. The tiers exist so that everyone can show what they can do, and the strongest teams can pull ahead.

2.5 What a good result looks like

Anyone can judge a resized design by eye, and the qualities that make one good are the same ones our scoring rewards:

- **No unwanted collisions** — overlaps are fine when they're intentional (layered graphics, deliberate stacking); what matters is that elements that shouldn't overlap, don't, and text never sits on a busy image.
- **Everything is readable** — text is large enough and has enough contrast; nothing is cut off.
- **It stays on brand** — colours, fonts, and the logo survive the resize intact and in proportion.
- **It looks intentional** — things line up, spacing is even, the most important element still stands out, pictures keep their proportions.
- **It holds up everywhere** — good across many different designs and shapes, not just one lucky example.

3. How the challenge runs

There are two stages: an automated first round that everyone enters and that narrows the field, and a live, in-person finale for the strongest teams.

3.1 Stage one — the engine (automated)

Every team submits its engine. We run each engine against a large set of hidden designs and target sizes it has never seen, across all three difficulty levels, and score every result automatically (see Section 4). The top six or seven teams advance.

Stage one runs on the team's own machine, offline. Teams may include and use a locally hosted model. To keep the field level, there is a limit on model size and a list of permitted open models; those specifics are being set with the team. Running locally within a shared limit means no team wins simply by calling a bigger outside service.

3.2 A short session with Adobe engineering

Teams that advance from stage one have a short session with members of the Adobe engineering team, where they walk through their approach – what they built, the decisions behind it, and any trade-offs made along the way. This happens before the live finale.

3.3 Stage two — the live finale

The finalists come to the Adobe Founders office and turn their engine into a full Adobe Express add-on that a real person would enjoy using. They will almost certainly keep improving the engine itself here, as building the real experience surfaces new needs. The finale is judged by a cross-functional jury – research, product, design, and engineering.

The finale is deliberately open-ended. A great resize is the core, but this problem has real room for product thinking, and we want teams to surprise us. Some directions a team might explore – offering several options to choose from, letting a user steer or lock parts of a layout, letting less-important elements gracefully drop away when space is tight – but none of these are required. A judge may well prefer a delightful experience with a slightly weaker engine on some edge cases over a strong engine with a flat experience.

Creativity is broader than fitting elements. The most inventive work is often in how a design is adapted — not only repositioning elements, but reflowing them, rewriting copy to fit, even regenerating imagery — all in service of keeping the brand's focus intact, not merely making things fit.

4. Rules and constraints

- Stage one runs offline, on the team's own machine. A locally hosted model is welcome and appropriate; there is simply a shared size limit and an approved-model list to keep things even.
- The engine ships in a standard container so it builds and runs the same way for everyone during scoring.
- Fixed materials may be relied upon — the brand's colours, fonts, and logo, and the catalogue of common canvas sizes.
- Solutions may not be hand-tailored to specific cases. Anything tuned to one design or one size will fail the hidden and unseen cases by design.

The exact model list, size limit, and container details are being finalized with the science team.

5. What to submit

5.1 Deliverables

- Stage one: the engine, with a short read-me explaining the approach and how to run it.
- Finale: the full Express add-on running on the engine, a short pitch, and a brief demo video

6. Glossary

A quick reference for terms used above.

Term	Meaning
Aspect ratio	The shape of a canvas – its width compared with its height. A square is 1:1; a phone story is tall; a web banner is very wide.
Resize / retarget	Producing a version of a design that fits a different canvas shape.
Reflow	Rearranging a design's pieces to suit a new shape, rather than stretching a fixed picture.
Element	A single piece of a design – a text, an image, or a shape.
Engine / service	A small program with no screen that takes input and returns output, so it can be tested automatically.
Add-on	A small tool that runs inside Adobe Express — what finalists turn their engine into a full version of at the live finale.
Renderer	A program that draws a design onto a canvas from instructions, placing the real assets – it does not invent imagery.
Multimodal / vision model	A model that can look at a picture (not just read text) — useful for interpreting a design.
Rubric	The written set of criteria the automated scorer follows.

7. How to Get Started

This section collects the starter kit's repo layout, submission rules, and FAQ in one place, for reference alongside the standalone versions that ship with the repo.

7.1 How to Get Started

Public repo: github.com/jhaaj08/chameleon — clone this to get everything below.

```
chameleon-starter/
├── Dockerfile
├── README.md
├── RULES.md
├── requirements.txt
├── chameleon_design_schema_v0.9.json
├── engine/
│   ├── main.py          <- orchestrator; you shouldn't need to edit this
│   └── resize.py        <- the only file you edit
├── brand_kit/
│   ├── assets/          <- backgrounds, logo, product art (asset_id targets)
│   └── fonts/           <- Solstice Serif / Solstice Sans
├── input/                <- drop task JSONs here (3 samples included)
└── output/               <- results land here
```

1. Clone the starter repo

```
git clone https://github.com/jhaaj08/chameleon.git
cd chameleon
```

2. Install dependencies

```
pip install -r requirements.txt
```

Requires Python 3.10+. This also installs cr-renderer (the real rendering engine, pulled straight from its public repo) — no separate setup needed.

3. Run it out of the box

```
python -m engine.main
```

This picks up every task JSON in input/ (3 sample designs are already there, one per difficulty level), and for each one writes output/<task>_output.json (the resized design) and output/<task>_output.png (a real rendered preview — actual fonts/assets, not placeholder boxes).

Out of the box, the engine uses a naive uniform-stretch baseline — deliberately the wrong approach. It's there so the pipeline runs on day one. Open output/*_output.png and you'll immediately see why stretching isn't good enough — that's the point.

4. Implement your approach

The only file you need to edit is engine/resize.py:

```
def resize(source: dict, target_canvas: dict) -> dict:
    ...
```

Everything else — input loading, the renderer, output writing, Docker packaging — is plumbing you don't need to touch. The full field-by-field spec is in chameleon_design_schema_v0.9.json; hard requirements (element ids preserved, nothing off-canvas, no regenerating image content, etc.) are in RULES.md.

5. Sanity-check before submitting

engine/main.py runs a quick structural self-check after every task (missing fields, off-canvas elements, mismatched ids) — a local sanity net, not the official scorer. A warning doesn't block output from being written; just decide whether it's a real problem.

6. Test in Docker — this is how you'll actually be run

```
docker build --platform linux/amd64 -t my-chameleon-engine .
docker run --rm --network none \
  -v $(pwd)/input:/app/input -v $(pwd)/output:/app/output \
  my-chameleon-engine
```

--network none matches stage-one scoring exactly: everything your engine needs must be installable at build time (add it to requirements.txt); nothing can be fetched at run time. If you use a locally hosted model, bake its weights into the image during the build step.

7.2 Submission rules

Supplements the rules above (Sections 4–5) with specific clarifications on what you submit and how it's judged. The standalone version is RULES.md in the starter repo.

Performance

Latency is not a disqualifying concern. There is no hard execution-time limit for stage one. A faster engine is preferred, all else equal — "speed and reliability" is a rubric row — but a slower, higher-quality result will not be thrown out just for being slower. Optimize for quality first.

What gets compared during scoring

Both artifacts your engine produces are evaluated, not just one:

- **The output JSON itself** — checked structurally: same element ids as the source (nothing added or removed), required fields present, nothing off-canvas, images kept in proportion, and so on.
- **The rendered PNG** — your output JSON run through the official renderer, then scored by the AI evaluator against the rubric.

A submission with plausible-looking JSON that doesn't actually render legibly doesn't get credit for the JSON, and a rendered image that happens to look fine despite a broken or dishonest JSON doesn't get credit for the image. Both are checked.

Models and fine-tuning

If your engine relies on a fine-tuned or adapted model — for example, a LoRA adapter on top of an open base model — the weights must be publicly accessible: hosted somewhere judges can pull them (e.g. a public Hugging Face repo), not kept private or shared ad hoc. Same spirit as the offline/size-limit rule above — every team's approach needs to be independently reproducible on the same footing.

Evaluation hardware

Stage-one scoring runs on a reference machine: **Apple M5 Pro** (*exact CPU/RAM/disk limits to be confirmed*). Build and test your engine within that budget — a submission that only runs comfortably on a machine with more headroom than the reference spec risks failing or timing out at actual scoring time.

Target canvas sizes

Target sizes used in scoring are drawn from a fixed catalogue of common real-world formats, described the way a designer would — e.g. "Instagram Post," "YouTube Thumbnail" — alongside their exact resolution and aspect

ratio. This catalogue is one of the "fixed materials" you're allowed to rely on (Section 5): you can safely assume target sizes will look like real platform formats, not arbitrary rectangles, even though the specific hidden targets used at scoring time are ones you won't have seen in advance.

Adaptive typography and copy — bonus, not required

Changing **font size** to help text fit its box is expected — that's core to the problem itself. Changing the **font family**, or rewriting/shortening the actual text content, is a different and much harder thing to judge automatically — it touches "faithfulness to the original design's intent." For stage-one automated scoring, treat content and font family as fixed: resize and reposition, don't rewrite. Adaptive typography and copy is exactly the kind of "creativity beyond fitting elements" called out for the finale (Section 3.3) — a strong direction to explore there, not something stage-one's automated rubric rewards.

Disqualifying: regenerating image content

Do not regenerate, redraw, or replace the pixel content of any asset — including with generative AI — even for a background or a minor decorative element. The engine repositions and resizes the elements it's given; it does not invent new imagery. This is a disqualifying violation, not a point deduction. If you're ever unsure whether something you're doing counts as "regenerating" versus "resizing," ask before you submit.

7.3 FAQ

The standalone version is CHAMELEON_FAQ.md in the starter repo.

Input & Output

Q1. What is the input?

The engine receives a single JSON object with two parts:

- `source` — the design you're resizing, at its original size: a canvas (width/height) plus a list of elements. Every element is one of three types — text, image, or shape — and carries a position, size, and type-specific properties. There are no semantic labels telling you which element is "the headline" or "the logo" — you get id, type, and geometry, same as the actual engine will at evaluation time.
- `target_canvas` — the width and height you need to resize the design into.

Your engine's job is to return the same elements — same ids, same count, nothing added or removed — with updated position/size (and, for text, optionally font size) so the design fits `target_canvas`.

The full field-by-field spec is in `chameleon_design_schema_v0.9.json`. Three worked examples, one per difficulty level, ship in the starter repo's `input/` folder:

File	Elements	Resize
chameleon_sample_level1_warmup.json	4	1080×1080 → 1080×1350
chameleon_sample_level2_core.json	11	1080×1080 → 1600×400
chameleon_sample_level3_advanced.json	17	1080×1350 → 728×90

Q2. Why can't we give the JSON in Crello's format, since you're using cr-renderer to render it?

Because Crello's format is a research dataset's internal storage layout, not something built for a hackathon spec to read:

- Coordinates are normalized [0,1] fractions of the canvas, not pixels.
- Categorical fields (element type, font, etc.) are stored as integer indices — you can't read them without the dataset's features object to decode them.
- Images are embedded, pre-rendered pixel crops baked into the data, not a reference to an asset file.
- Rotation is stored in units of π, not degrees.

That's plumbing specific to how CyberAgent happens to store the Crello dataset — it has nothing to do with the actual resize problem. Our schema keeps pixel coordinates, plain field names, and simple `asset_id` references so the design problem is what you're reading, not a dataset format. cr-renderer is still doing the real rendering work under the hood — we convert our schema into Crello's format right before the render call. That conversion is on us, not something you need to touch.

Rendering & local preview

Q3. How do I render my output to see what it actually looks like?

You don't need a separate tool — `python -m engine.main` (Section 9.1, step 3) renders every task through cr-renderer, installed straight from its public repo as part of `pip install -r requirements.txt`. Run the starter kit out of the box and `output/*_output.png` is already a real rendered preview: actual brand fonts, actual assets — not placeholder boxes.

It handles text wrapping at real glyph widths, rotated elements, shape elements rasterized before rendering (Crello has no native shape primitive), and missing assets shown as a placeholder rather than a crash — try the Level 3 sample's intentionally-missing `holiday_watermark` asset to see this for yourself. Requires Python 3.10+.

Q4. Where do I find the actual brand assets and fonts?

- `brand_kit/assets/` — the image files referenced by `asset_id` in the sample JSONs: backgrounds, logo, product art.
- `brand_kit/fonts/` — the font files matching the `font_family` values used in the schema (`Solstice Serif`, `Solstice Sans`).

`engine/main.py` resolves both automatically when it renders your output (Q3) — you don't need to point anything at them yourself. Just make sure any asset/font-loading code in your own engine or viewer resolves paths against these same two folders. One deliberate exception: the Level 3 sample's `holiday_watermark` element references `solstice_holiday_badge_2025.png`, which is **not** in `brand_kit/assets/` — that's the intentional missing-asset case from Q1, not a packaging gap. Don't add it.

Using the starter kit & submitting

Q5. How do I use the starter kit, and how do I submit?

See Section 9.1 above for the full step-by-step walkthrough (clone → install → run → implement `resize()` → self-check → Docker).

Submitting: your stage-one submission is your engine — code, Dockerfile, and a short read-me explaining your approach and how to run it — submitted before code freeze. Where to send it, and any submission form, will be announced separately ahead of the deadline.